```java
import java.util.*;

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public int getAge() { return age; }
}

public class AverageAge {

    public static void main(String[] args) {

        List<Person> people = Arrays.asList(
                new Person("A", 20),
                new Person("B", 30),
                new Person("C", 40)
        );

        double avg = people.stream()
                .mapToInt(Person::getAge)
                .average()
                .orElse(0.0);

        System.out.println("Average Age: " + avg);
    }
}

```

```java
import java.util.*;
import java.util.stream.Collectors;

public class GroupByLength {

    public static void main(String[] args) {

        List<String> words = Arrays.asList("apple","bat","ball","cat","banana","dog","goat");

        Map<Integer, List<String>> result = words.stream()
                .collect(Collectors.groupingBy(String::length));

        System.out.println("Grouped by Length: " + result);
    }
}
```

```java
//  1. Second Highest Number

import java.util.*;

public class SecondHighest {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(5, 9, 11, 2, 8, 21, 1);

        int result = numbers.stream()
                .distinct() // handle duplicates
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst()
                .orElse(-1);

        System.out.println("Second Highest: " + result);
    }
}
```

```java
//2. Sort Employees by Salary

import java.util.*;

class Employee {
    int id;
    String name;
    double salary;
    String department;

    Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public double getSalary() { return salary; }
    public String getDepartment() { return department; }

    public String toString() {
        return name + " : " + salary;
    }
}

public class SortEmployees {

    public static void main(String[] args) {

        List<Employee> employees = Arrays.asList(
                new Employee(101, "John", "HR", 50000),
                new Employee(102, "Alice", "IT", 70000),
                new Employee(103, "Bob", "IT", 45000),
                new Employee(104, "David", "Finance", 90000)
        );

        List<Employee> asc = employees.stream()
                .sorted(Comparator.comparing(Employee::getSalary))
                .toList();

        List<Employee> desc = employees.stream()
                .sorted(Comparator.comparing(Employee::getSalary).reversed())
                .toList();

        System.out.println("Ascending: " + asc);
        System.out.println("Descending: " + desc);
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

class EmployeeAvg {
    int id;
    String name;
    String department;
    double salary;

    EmployeeAvg(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public String getDepartment() { return department; }
    public double getSalary() { return salary; }
}

public class AvgSalaryByDept {

    public static void main(String[] args) {

        List<EmployeeAvg> employees = Arrays.asList(
                new EmployeeAvg(1, "John", "HR", 50000),
                new EmployeeAvg(2, "Jane", "IT", 70000),
                new EmployeeAvg(3, "Mike", "IT", 80000),
                new EmployeeAvg(4, "Sara", "Finance", 60000),
                new EmployeeAvg(5, "Paul", "HR", 55000)
        );

        Map<String, Double> result = employees.stream()
                .collect(Collectors.groupingBy(
                        EmployeeAvg::getDepartment,
                        Collectors.averagingDouble(EmployeeAvg::getSalary)
                ));

        System.out.println("Average Salary by Dept: " + result);
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

public class DeptMoreThanTwo {

    public static void main(String[] args) {

        List<EmployeeAvg> employees = Arrays.asList(
                new EmployeeAvg(1, "John", "IT", 60000),
                new EmployeeAvg(2, "Alice", "HR", 55000),
                new EmployeeAvg(3, "Bob", "IT", 75000),
                new EmployeeAvg(4, "Carol", "HR", 70000),
                new EmployeeAvg(5, "David", "Finance", 65000),
                new EmployeeAvg(6, "Eve", "IT", 80000),
                new EmployeeAvg(7, "Frank", "Finance", 64000),
                new EmployeeAvg(8, "Ala", "Operations", 44000)
        );

        Map<String, Long> map = employees.stream()
                .collect(Collectors.groupingBy(
                        EmployeeAvg::getDepartment,
                        Collectors.counting()
                ));

        List<String> result = map.entrySet().stream()
                .filter(e -> e.getValue() > 2)
                .map(Map.Entry::getKey)
                .toList();

        System.out.println("Departments with >2 employees: " + result);
    }
}
```

```java
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class CountOccurrences {

    public static void main(String[] args) {

        List<String> items = Arrays.asList("apple","banana","apple","orange","banana");

        Map<String, Long> map = items.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ));

        System.out.println("Count: " + map);
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

public class PartitionEvenOdd {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8,9,10);

        Map<Boolean, List<Integer>> result = numbers.stream()
                .collect(Collectors.partitioningBy(n -> n % 2 == 0));

        System.out.println("Even: " + result.get(true));
        System.out.println("Odd: " + result.get(false));
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

public class HighestPaidByDept {

    public static void main(String[] args) {

        List<EmployeeAvg> employees = Arrays.asList(
                new EmployeeAvg(1, "John", "HR", 50000),
                new EmployeeAvg(2, "Jane", "IT", 70000),
                new EmployeeAvg(3, "Mike", "IT", 80000),
                new EmployeeAvg(4, "Sara", "Finance", 60000),
                new EmployeeAvg(5, "Paul", "HR", 55000)
        );

        Map<String, EmployeeAvg> result = employees.stream()
                .collect(Collectors.groupingBy(
                        EmployeeAvg::getDepartment,
                        Collectors.collectingAndThen(
                                Collectors.maxBy(Comparator.comparing(EmployeeAvg::getSalary)),
                                Optional::get
                        )
                ));

        result.forEach((dept, emp) ->
                System.out.println(dept + " -> " + emp.name + " : " + emp.salary));
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

public class HighestAvgSalaryDept {

    public static void main(String[] args) {

        List<EmployeeAvg> employees = Arrays.asList(
                new EmployeeAvg(1, "John", "IT", 60000),
                new EmployeeAvg(2, "Alice", "HR", 55000),
                new EmployeeAvg(3, "Bob", "IT", 75000),
                new EmployeeAvg(4, "Carol", "HR", 70000),
                new EmployeeAvg(5, "David", "Finance", 65000),
                new EmployeeAvg(6, "Eve", "IT", 80000),
                new EmployeeAvg(7, "Frank", "Finance", 64000),
                new EmployeeAvg(8, "Ala", "Operations", 44000)
        );

        Map<String, Double> avgMap = employees.stream()
                .collect(Collectors.groupingBy(
                        EmployeeAvg::getDepartment,
                        Collectors.averagingDouble(EmployeeAvg::getSalary)
                ));

        Map.Entry<String, Double> result = avgMap.entrySet().stream()
                .max(Map.Entry.comparingByValue())
                .orElseThrow();

        System.out.println("Highest Avg Salary Dept: " + result);
    }
}
```